2025 IEEE International Conference on Communications Workshops (ICC Workshops) | 979-8-3315-9624-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCWorkshops67674.2025.11162423

# Energy-Efficient Collaborative AI Service Migration and Resource Allocation Based on Traffic Prediction in 6G Networks

Hongling Zhou[‡,§], Qin Li[†], Nanxiang Shi[†], Yuhao Chai[‡,§], Yong Zhang[‡,§*], Yinglei Teng[‡,§]
[‡]School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China
[§]Beijing Key Laboratory of Work Safety Intelligent Monitoring, Beijing University of Posts and Telecommunications, Beijing, China
[†]Department of Basic Network Technology Research, China Mobile Research Institute, Beijing, China
{zhouhongling, chaiyh}@bupt.edu.cn
{liqinyjy, shinanxiang}@chinamobile.com
{lilytengtt}@gmail.com

***Abstract*—As a more advanced next-generation mobile communication system, sixth-generation (6G) will be tightly integrated with Artificial Intelligence (AI) technologies and will combine cloud-network-edge collaborative computing architecture to provide diverse AI services. However, due to the time-varying nature of network traffic, ensuring Quality of Service (QoS) and continuity for AI services remains a challenge. To address this issue, we focus on the migration of AI services in temporal dynamic traffic scenarios, and model it as an optimization problem. We aim at minimizing the long-term average system energy consumption, and propose a collaborative agent migration with deep neural network (DNN) deployment and dynamic bandwidth resource allocation algorithm based on traffic prediction (CAMD-TP). DLinear is used to predict the time-varying network traffic for AI service migration decisions. We apply Lyapunov optimization to convert the long-term stochastic optimization problem into a short-term deterministic problem. We propose an optimization algorithm based on the general Benders decomposition (GBD) technology to solve the AI service migration and dynamic bandwidth resource allocation problems. Experimental results show that, compared with two heuristic algorithms, CAMD-TP reduces the average system energy consumption up to 18.7%, and it outperforms existing algorithms in both traffic acceptance rate and throughput.**

***Index Terms*—Artificial intelligence service migration, quality of AI service, DNN inference, network traffic prediction, lyapunov optimization**

## I. Introduction

With the advancement of network technologies, the integration of AI with the upcoming 6G networks has become an inevitable trend, aiming to provide more diverse AI services such as immersive interactions, holographic communication. With the support of Network Function Virtualization (NFV) technology, traditional network services are implemented through service function chains (SFC) composed of a series of virtual network functions (VNF). AI services rely on DNN models that can provide more intelligent decisions.

*Corresponding author: yongzhang@bupt.edu.cn

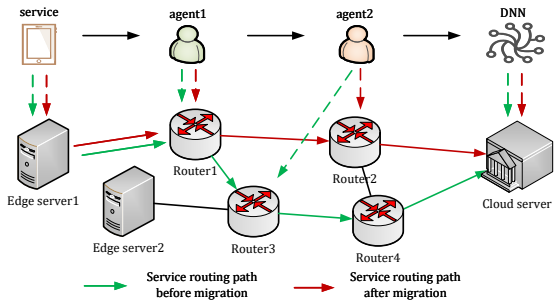



Fig. 1. AI Service Migration scenario

By leveraging the advantages of both, AI services can be decomposed into multiple functional modules, deployed as multiple agents for preprocessing, and finally reasoned by DNN for intelligent decision-making [1]. This structure realizes a flexible and efficient processing procedure in the form of an AI service chain. Agents can be traditional application-layer network functions, such as firewall (FW) and switch, or AI-driven network functions, such as intelligent root cause analysis and intelligent anomaly detection. While 6G offers ultra-high network throughput, it also increases the complexity of dynamic traffic management.In the case of traffic fluctuation, the fixed service chain deployment scheme will lead to low resource utilisation, increased latency, increased service failure rate, and excessive system energy consumption. As shown in Figure. 1, due to the fluctuation of traffic, the static layout fails to meet the QoS requirements. To address this issue, migration is employed to ensure the QoS and continuity of AI services, which can effectively adapt to the traffic fluctuation and improve the flexibility of the system.

Most existing research on VNF deployment and SFC migration is still focused on cloud data center scenarios [2], [3]. Meanwhile, DNN inference services are usually placed in



Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:16:05 UTC from IEEE Xplore. Restrictions apply.

the cloud , suffering from high latency [3], or at the edge, which lacks sufficient computational resources. Therefore, to meet the diverse service requirements, there is an urgent need to study AI service chain migration in cloud-network-edge scenarios. Currently, most research focuses on the migration of traditional VNFs, primarily addressing node or link overload and failure [2], with limited consideration for the dynamic characteristics of services. Some studies attempt to orchestrate VNFs in advance by predicting dynamic changes in network traffic and resource demands [4], but the unique requirements of AI services and DNN deployment have not been fully explored. By utilizing traffic prediction methods, the changing trends of service traffic can be predicted in advance, enabling effective scheduling before traffic fluctuations occur and ensuring the continuity of services. However, traffic prediction has limitations, including constrained prediction accuracy, reliance on large datasets, and high computational costs [4]. To address these challenges, we utilize DLinear [5] for accurate and fast network traffic prediction to assist AI service chain migration decisions. DLinear is a simple linear model with relatively few parameters, allowing for rapid training and prediction.

This paper focuses on the migration of AI services in temporal dynamic traffic scenarios. The main contributions of this paper are summarized as follows.

- Under the constraints of service latency, inference accuracy, and limited network resources, we formulate a long-term average system energy consumption minimization problem for AI service chain migration in cloud-network-edge collaborative scenarios. To solve this, we propose a collaborative agent migration with DNN deployment and dynamic bandwidth resource allocation algorithm based on traffic prediction (CAMD-TP).
- Incorporating the service traffic prediction results from DLinear, we apply Lyapunov optimization technique to transform the long-term optimization problem into a short-term deterministic one. The integration of these two approaches enables better balancing of system energy consumption and queue stability.
- Considering the mixed-integer linear programming characteristics of the optimization problem, we design a joint AI service migration and resource allocation scheme based on GBD technique. Experimental results show that, compared with two heuristic algorithms, our algorithm reduces the average system energy consumption by 18.7% and outperforms the existing algorithms in both traffic acceptance rate and throughput.

## II. NETWORK MODEL AND PROBLEM FORMULATION

### A. Network model

The migration scenario of AI service chains in 6G networks is shown in Figure. 1. It consists of remote cloud servers, base stations equipped with edge servers, and network routing nodes integrated with computing capabilities. We consider time-varying service requests in the system.

The graph $G(V, E)$ represents the actual network, where $V$ and $E$ denote the sets of nodes and links, respectively. Each node $v \in V$ has different computation capacity $C_v^{cpu}$ and memory size $C_v^{mem}$. Each link $uv \in V$ connecting node $u$ and $v$ is defined by its total bandwidth $C_{uv}^{link}$.

The set of AI users is represented by $\mathcal{I} = \{1, 2, \ldots, I\}$. For AI user $i \in \mathcal{I}$, each service is defined as $W_i(t) = \{s_i, o_i, A_i(t), R_i, \eta_i, D_i, N_i\}$, where $s_i$ and $o_i$ represent the source and destination node, respectively. $A_i(t)$ represents the arrival rate of user's traffic data. $R_i$ indicates user's minimum transmission rate requirement. $\eta_i$ indicates the inference accuracy requirement. $D_i$ is the maximum tolerated delay. Let $N_i = \left\{n_{i,1}, n_{i,2}, ..., n_{i,|N_i|}\right\}$ be the order set of agents. To ensure the inference accuracy of AI services, each service must select an appropriate DNN model $m \in M = \{1, 2, ..., |M|\}$ to complete the inference service, where $M$ represents the set of DNN models. Since AI services are processed sequentially through multiple agents and DNN model, we define an extended sequence set of agents $N_i^{'} = \left\{n_{i,0}, n_{i,1}, n_{i,2}, \ldots n_{i,|N_i|}, n_i^m\right\}$, where $n_{i,0}$ represents the virtual agent deployed on the source node $s_i$, and $n_i^m$ represents the DNN model $m$ deployed on the destination node $o_i$.

Assume that time is discretized into time slots. Let $\mathcal{T} = \{1, \ldots, t, \ldots, T\}$ represents the time slot system. $x_v^{i,n}(t) = 1$ denotes that agent $n$ of service $i$ is deployed on node $v$ at time $t$. $y_{uv}^{i,nn^{'}}(t) = 1$ represents that service $i$ flow passes through the node $u$ carrying agent $n$ and routes to the node $v$ carrying the next agent $n'$. Define $\delta_n^i(t) = 1$ represents agent $n$ for service $i$ migrates at time $t$, and $\delta_n^i(t) = 0$ otherwise. $r_i(t)$ is the link bandwidth allocated to user $i$.

### B. Problem Formulation

To ensure the QoS requirements of services under traffic fluctuations, we consider constraints on network resources, delay and inference accuracy, proposing an optimization problem aimed at minimizing the average system energy consumption. To ensure the effectiveness of the migration strategy, a new migration strategy is executed for each AI service at every discrete time slot $t$. The corresponding formula constraints are as follows.

*Agent placement constraint*:

$$C1: \sum_{v \in V} x_v^{i,n}(t) = 1, \forall n \in N_i^{'}, \forall i \in \mathcal{I} \tag{1}$$

$$C2: \sum_{v \in V} y_{uv}^{i,nn^{'}}(t) - \sum_{v \in V} y_{vu}^{i,nn^{'}}(t) = x_u^{i,n}(t) - x_u^{i,n^{'}}(t), \quad \forall i \in \mathcal{I}, \forall u, v \in V, \forall n, n^{'} \in N_i^{'} \tag{2}$$

$$C3: \sum_{uv \in E} y_{uv}^{i,nn^{'}}(t) \geq 1, \forall i \in \mathcal{I}, \forall n, n^{'} \in N_i^{'} \tag{3}$$

$C1$ indicates that agent $n$ of service $i$ can only be mapped to one physical node. $C2$ ensures that there must be a continuous path in the physical network for the flow between adjacent agents of service $i$ [6]. To simplify the problem, we ensure that the flow between adjacent agents should be mapped to at least one physical link by defining $C3$.



Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:16:05 UTC from IEEE Xplore. Restrictions apply.

*Node and link capacity constraints*: The computation and memory consumption of agent $n$ for service $i$ is represented as $\gamma_{i,n}^{cpu}$ and $\gamma_{i,n}^{mem}$.

$$C4: \sum_{i\in\mathcal{I}} \sum_{n\in N_i'} x_v^{i,n}(t)\gamma_{i,n}^{cpu} \le C_v^{cpu}, \forall v \in V \tag{4}$$

$$C5: \sum_{i\in\mathcal{I}} \sum_{n\in N_i'} x_v^{i,n}(t)\gamma_{i,n}^{mem} \le C_v^{mem}, \forall v \in V \tag{5}$$

$$C6: \sum_{i\in\mathcal{I}} \sum_{n\in N_i'} y_{uv}^{i,nn'}(t) r_i(t) \le C_{uv}^{link}, \forall uv \in E \tag{6}$$

where $C4$, $C5$, and $C6$ represent the CPU and memory resource limitations of physical nodes, as well as the bandwidth limitations of links, respectively.

*Service bandwidth constraint*: $C7$ denotes the bandwidth requirement of the service.

$$C7: r_i(t) \ge R_i, \forall i \in \mathcal{I} \tag{7}$$

*Service migration constraint*: We assume an initial deployment scenario where all services have already been deployed in the network. A migration is indicated when an agent is mapped to different nodes in two consecutive time slots, i.e.,

$$\begin{aligned} C8: \delta_n^i(t) \ge x_v^{i,n}(t) - x_v^{i,n}(t-1), \\ \forall i \in \mathcal{I}, \forall n \in N_i, \forall v \in V \end{aligned} \tag{8}$$

*Inference accuracy constraint*: We specify that the DNN model required for AI services is stored in the server and activated immediately upon detection of service demand. Considering the AI characteristics of certain agents, let $\eta_n^i$ denote the inference accuracy of agent $n$ for service $i$. $\eta^m$ represents the inference accuracy level of DNN model $m$. Define $h_i\left(\eta_m, \left\{\eta_n^i \mid \forall n \in N_i\right\}\right)$ as the inference accuracy of AI service $i$. Without loss of generality, the inference accuracy of an AI service is simplified as the product of the accuracy of each agent in the chain and the selected DNN model $m$. AI services' inference accuracy must meet the accuracy requirements, namely

$$C9: \prod_{n\in N_i} \eta_n^i \sum_{m\in M} x_v^{i,n_i^m}(t)\eta^m \ge \eta_i, \ \forall i \in \mathcal{I} \tag{9}$$

*Service delay constraint*: The total end-to-end latency of each AI service includes network transmission latency, agent processing latency, and DNN inference latency. For the migration process, we adopt the live migration method [7], where migration time is generally negligible when resources are sufficient. Since the service traffic that passes through different agents is not constant [1], let $\beta_n$ denote the traffic change factor through agent $n$ and the cumulative change rate of AI service $i$ can be expressed as $\rho_i = \prod_{n\in N_i} \beta_n$. The delay needs to meet the delay requirements, i.e.,

$$\begin{aligned} C10: \sum_{n,n'\in N_i'} \sum_{uv\in E} y_{uv}^{i,nn'}(t) l_{uv} \frac{\prod_{j=n_{i,0}}^{n} \beta_j A_i(t)}{r_i(t)} + \\ \sum_{n\in N_i'} \frac{\prod_{j=n_{i,0}}^{n-1} \beta_j A_i(t) L_{den}}{\gamma_{i,n}^{cpu}} \le D_i, \forall\ i \in \mathcal{I} \end{aligned} \tag{10}$$

where the number of hops between node $u$ and node $v$ in the network is $l_{uv}$ and $L_{den}$ represents the processing density.

*Queue constraint*: The system backlog queue for user $i$ at time slot $t$ can be expressed as

$$Q^i(t+1) = \max\left[Q^i(t) - r_i(t)T_0, 0\right] + \rho_i A_i(t) T_0 \tag{11}$$

$$Q^{AI}(t) = \left[Q^1(t), Q^2(t), \ldots, Q^I(t)\right] \tag{12}$$

where $T_0$ is the duration of the time interval. We cache the services that have been received but not yet processed in the form of a queue, i.e.,

$$\begin{aligned} Z^i(t+1) = \sum_{m\in M} \sum_{v\in V} x_v^{i,n_i^m}(t) \max\left[Z^i(t) - \gamma_{i,n_i^m}^{cpu}, 0\right] \\ + \min L_{den}\left[Q^i(t), r_i(t)\right] \end{aligned} \tag{13}$$

$$Z^{AI}(t) = \left[Z^1(t), Z^2(t), \ldots, Z^I(t)\right] \tag{14}$$

To ensure the long-term stability of the network, the constraints are described as follows

$$C11: \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} E\left\{Q^{AI}(t)\right\} < \infty \tag{15}$$

$$C12: \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} E\left\{Z^{AI}(t)\right\} < \infty \tag{16}$$

*Energy consumption*: The system energy consumption mainly considers the consolidation energy and the migration energy consumption, which characterize the energy consumed by agents running on the server and the energy consumed for memory transfer during migration, respectively. we assume a linear server energy consumption model [8]. Hence, the total energy consumption of the system can be expressed as

$$\begin{aligned} E_{tot}(t) = \sum_{v\in V} \sum_{i\in\mathcal{I}} \sum_{n\in N_i'} x_v^{i,n}(t)(1-\varepsilon) \frac{\gamma_{i,n}^{cpu}}{C_v^{cpu}} P^{max} T_0 + \\ \sum_{v\in V} \varepsilon P^{max} T_0 + \sum_{i\in\mathcal{I}} \sum_{n\in N_i} 2\delta_n^i(t)(1-\varepsilon) P^{max} \frac{\gamma_{i,n}^{mem}}{L_{agg}} t_{agg} \end{aligned} \tag{17}$$

where $P^{max}$ denotes the maximum server power consumption, $\varepsilon \in [0,1]$ is the ratio of the baseline power to the maximum power, $L_{agg}$ is the packet length of the stream carrying the memories, and $t_{agg}$ is the packet processing time during migration. Under fluctuating traffic, migration energy consumption greatly affects total energy consumption while ensuring AI service QoS, requiring a suitable migration strategy to balance energy and performance.

Based on the above analysis, the AI service migration problem can be expressed as follows:

$$\begin{aligned} \mathcal{P}0: \min_{\{\Psi,\mathbb{R}\}} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} E_{tot}(t) \\ \text{s.t.} \quad C1 - C12 \end{aligned} \tag{18}$$

where $\Psi$ represents the sets of $x_v^{i,n}(t)$, $y_{uv}^{i,nn'}(t)$, and $\delta_n^i(t)$, and $\mathbb{R}$ represent the set of $r_i(t)$. The objective function is a long-term optimization problem, making it challenging to



Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:16:05 UTC from IEEE Xplore. Restrictions apply.

obtain an optimal solution directly. To address this issue, we first utilize DLinear to predict the service flow for the next moment. Then we apply Lyapunov optimization technique to transform it into a short-term deterministic problem and employ GBD technology to determine migration decisions and resource allocation.

## III. PROBLEM TRANSFORMATION AND ALGORITHM DESIGN

### A. AI Service Traffic Prediction

Time series forecasting aids in obtaining future information in advance. Among existing approaches, DLinear [5] is a simple yet efficient linear prediction model. DLinear decomposes the original data into trend and seasonal vectors, which are then fed into corresponding single-layer linear layers. The two trends are summed to yield the prediction result. This model focuses on modeling linear relationships, making it well suited for time series with strong linear trends and seasonality, effectively capturing periodic features in service flow. Additionally, it has a lower computational complexity, enabling rapid generation of predictions, which helps to quickly make migration decisions in the next stage and complete the migration services in time.

### B. Problem Transformation Using Lyapunov Optimization

In this paper, the prediction-based AI service migration algorithm only considers the prediction results for the next time step. Let $A_i^*(t)$ represents the arrival rate of the flow data for service $i$ at the predicted time slot $t$. Let $\Theta(t) = \left[Q^{AI}(t), Z^{AI}(t)\right]$, the Lyapunov function is defined as $L(\Theta(t)) = \frac{1}{2}\left\{\sum_{i\in\mathcal{I}}\left[Q^i(t)\right]^2 + \sum_{i\in\mathcal{I}}\left[Z^i(t)\right]^2\right\}$.

The one-slot Lyapunov drift is given by$\Delta(\Theta(t)) = \mathbb{E}\left\{L(\Theta(t+1)) - L(\Theta(t))|\Theta(t)\right\}$. The drift minimization method uses the current service state and the current queue backlog to stabilize the system. Accordingly, the drift-plus-penalty expression can be expressed as $\Delta(\Theta(t)) + V\mathbb{E}\left\{E_{tot}(t)|\Theta(t)\right\}$, where $V$ is a non-negative control parameter used to represent the trade-off between the total energy consumption and the Lyapunov drift. The drift-plus-penalty function is is upper bounded by

$$\begin{aligned}
&\Delta(\Theta(t)) + V\mathbb{E}\left\{E_{tot}(t)|\Theta(t)\right\} \leq \\
&C - \sum_{i\in\mathcal{I}} Q^i(t)\mathbb{E}\left[r_i(t)T_0 - \rho_i A_i^*(t)T_0|\Theta(t)\right] \\
&- \sum_{i\in\mathcal{I}}\sum_{m\in M}\sum_{v\in V} Z^i(t)\mathbb{E}\left\{x_v^{i,n_i^m}(t)\left[\gamma_{i,n_i^m}^{cpu} - L_{den}r_i(t)\right]\Big|\Theta(t)\right\} \\
&+ V\mathbb{E}\left\{E_{tot}(t)|\Theta(t)\right\} \\
&= C + \sum_{i\in\mathcal{I}} Q^i(t)\rho_i A_i^*(t)T_0 + \mathcal{F}(\Psi,\mathbb{R})
\end{aligned} \tag{19}$$

which satisfies the following constraint

$$\begin{aligned}
C \geq \frac{1}{2}\Bigg\{&\left[\sum_{i\in\mathcal{I}} r_i(t)T_0\right]^2 + \left[\sum_{i\in\mathcal{I}}\rho_i A_i^*(t)T_0\right]^2 \\
&+ \left[\sum_{i\in\mathcal{I}}\sum_{m\in M}\sum_{v\in V} x_v^{i,n_i^m}(t)\gamma_{i,n_i^m}^{cpu}\right]^2 \\
&+ \left[\sum_{i\in\mathcal{I}}\sum_{m\in M}\sum_{v\in V} x_v^{i,n_i^m}(t)L_{den}r_i(t)\right]^2\Bigg\}
\end{aligned} \tag{20}$$

The original problem $\mathcal{P}0$ can be transformed into $\mathcal{P}1$:

$$\begin{aligned}
\mathcal{P}1: &\min_{\{\Psi,\mathbb{R}\}} \mathcal{F}(\Psi,\mathbb{R}) = V\left\{E_{tot}(t)\right\} - \sum_{i\in\mathcal{I}} Q^i(t)\mathbb{E}\left[r_i(t)T_0\right] \\
&- \sum_{i\in\mathcal{I}}\sum_{m\in M}\sum_{v\in V} Z^i(t)\mathbb{E}\left\{x_v^{in_i^m}(t)\left[\gamma_{i,n_i^m}^{cpu}(t) - L_{den}r_i(t)\right]\right\} \\
&\text{s.t.} \quad C1 - C12
\end{aligned} \tag{21}$$

### C. Optimal Service Migration and Dynamic Bandwidth Resource Allocation Algorithm Based on GBD

Problem $\mathcal{P}1$ is an NP-hard mixed integer linear programming problem. By observation, we can determine that if the binary variable $\Psi$ are fixed , the remaining components involving $\mathbb{R}$ form a convex optimization problem, which can be effectively addressed using the GBD method [9]. The derivation process of GBD is as follows.

At iteration $w$, by fixing the binary variables $\Psi^{(w)}$ in the original problem $\mathcal{P}1$, the primal problem $\mathcal{P}2.1$ can be expressed as

$$\begin{aligned}
&\mathcal{P}2.1: \min_{\{\mathbb{R}\}} \mathcal{F}\left(\Psi^{(w)},\mathbb{R}\right) \\
&\text{s.t.} \quad C6^{'}: \sum_{i\in\mathcal{I}}\sum_{n\in N_i'} y_{uv}^{i,nn^{'}(w)}(t)r_i(t) \leq C_{uv}^{link},\ C7 \\
&C10^{'}: \sum_{n,n^{'}\in N_i'}\sum_{uv\in E} y_{uv}^{i,nn^{'}(w)}(t)l_{uv}\frac{\prod_{j=n_{i,0}}^{n}\beta_j A_i^*(t)}{r_i(t)} \\
&\qquad + \sum_{n\in N_i'}\frac{\prod_{j=n_{i,0}}^{n-1}\beta_j A_i^*(t)L_{den}}{\gamma_{i,n}^{cpu}} \leq D_i
\end{aligned} \tag{22}$$

All the constraints of problem $\mathcal{P}2.1$ can all be expressed in the form of $g_k\left(\Psi^{(w)},\mathbb{R}\right) \leq b_k$, where $k = 1,2,\ldots K$, and $K$ is the number of constraints in problem $\mathcal{P}2.1$.

If problem $\mathcal{P}2.1$ is feasible, its solution provides the optimal values $\mathbb{R}^{(w)}$ and an UB of problem $\mathcal{P}1$. Additionally, its solution also provides the corresponding Lagrange multiplier vector $\Lambda^{(w)} = \left\{\lambda_k^{(w)}, k = 1,2,\ldots K\right\}$, where $\lambda_k^{(w)}$ is the Lagrange multiplier for the $k$th constraint. The Lagrange function of $\mathcal{P}2.1$ can be formulated as

$$\mathcal{L}(\Psi,\mathbb{R}^{(w)},\Lambda^{(w)}) = \mathcal{F}(\Psi,\mathbb{R}^{(w)}) + \sum_{k=1}^{K}\lambda_k^{(w)}[g_k(\Psi,\mathbb{R}^{(w)}) - b_k] \tag{23}$$



Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:16:05 UTC from IEEE Xplore. Restrictions apply.

If problem $\mathcal{P}2.1$ is infeasible, we consider formulating an $l_1 - minimization$ problem [9] as follows:

$$\begin{aligned} &\mathcal{P}2.2 : \min_{\{\mathbb{R}\}} \sum_{k=1}^{K} \xi_k \\ \text{s.t.} \quad & g_k\left(\Psi^{(w)}, \mathbb{R}\right) \leq b_k + \xi_k, k = 1, 2, \ldots K \\ & \xi_k \geq 0, k = 1, 2, \ldots K \end{aligned} \tag{24}$$

where we define $\Xi = [\xi_k, k = 1, 2, \ldots K]$. When $\sum_{k}^{|K|} \xi_k = 0$, the primal problem is feasible; otherwise, the primal problem is infeasible. Let $\bar{\Lambda}^{(w)} = \left\{\bar{\lambda}_k^{(w)}, k = 12, ..., K\right\}$ be the corresponding Lagrange multiplier vector. The Lagrange function generated by the infeasible solution is defined as

$$\bar{\mathcal{L}}(\Psi, \mathbb{R}^{(w)}, \bar{\Lambda}^{(w)}) = \sum_{k=1}^{K} \bar{\lambda}_k^{(w)} [g_k(\Psi, \mathbb{R}^{(w)}) - b_k - \xi_k] \tag{25}$$

The relaxed master problem $\mathcal{P}2.3$ is represented as:

$$\begin{aligned} &\mathcal{P}2.3 : \min_{\{\Psi, \varpi\}} \varpi \\ \text{s.t.} \quad & C1 - C6, C8, \\ & C13 : \mathcal{L}(\Psi, \mathbb{R}^{(w)}, \Lambda^{(w)}) \leq \varpi, \forall w \in W \\ & C14 : \bar{\mathcal{L}}(\Psi, \mathbb{R}^{(w)}, \bar{\Lambda}^{(w)}) \leq 0, \forall w \in \bar{W} \end{aligned} \tag{26}$$

where $C13$ and $C14$ denote the feasible and infeasible cuts. $W$ denotes the set of iterations for the feasible primal problem, and $\bar{W}$ denotes the set of iterations for the infeasible primal problem. The master problem $\mathcal{P}2.3$ is a linear programming problem with the binary variables $\Psi$ and a scalar variable $\varpi$, whose solution provides the LB for the original problem $\mathcal{P}1$ and determines the binary variables $\Psi^{(w+1)}$ in the next iteration.The CAMD-TP algorithm is described in detail in Algorithm 1.

## IV. PERFORMANCE EVALUATION

We evaluate the performance of the proposed algorithm by simulations. The real network dataset [10] used in this study comes from the United Kingdom academic network backbone. We conduct the simulation using the Atlanta network [11], which consists of 15 nodes and 22 links. The simulation deploys 2 edge servers, 1 cloud server, and 4 computing nodes, all of which are equipped with memory and computing resources. Nodes without computing capabilities primarily perform service forwarding.In the simulation, we adopt image processing services as AI tasks, with each AI task consisting of 1-3 agents in a fixed sequence. We consider four types of agents, including firewall, format conversion, data analysis, and data augmentation. Their computation resource requirements are 400 MHz, 900 MHz, 1.1 GHz, and 1.2 GHz. The inference accuracy for each agent is 1, 1, 0.7, and 0.85, respectively.

### A. Prediction Performance

We compare DLinear with two other time series forecasting models. One is the Prophet model [12] developed by Facebook, which is a useful tool for predicting seasonal pattern data. The other is the Gated Recurrent Unit (GRU) [4], which is an improved version of recurrent neural networks. Table I presents a comparison of the predictive performance of three models, with the best results highlighted in bold. The data in Table I show that DLinear exhibits lower errors across all metrics, outperforming the other two models. Figure 2 provides a detailed comparison of the actual values against the predicted values generated by DLinear, confirming the excellent predictive capability of DLinear.

**Algorithm 1** CAMD-TP Algorithm

**Input:** $G, \mathcal{I}, W_i(t), C_v^{cpu}, C_v^{mem}, C_{uv}^{link}, \gamma_{i,n}^{cpu}, \gamma_{i,n}^{mem}, V$
**Output:** $\Psi, \mathbb{R}$

```
for t in T do
    for i in I do
        Predict the future traffic A_i^*(t) by DLinear;
    end for
    while |UB − LB| > ε do
        Solve the primal problem P2.1;
        if P2.1 (Ψ^(w)) is feasible then then
            Obtain the continuous solution R^(w) and the Langrange multiplier Λ^(w) and calculate the upper bound UB and the feasible cut;
        else if P2.1 (Ψ^(w)) is infeasible then then
            Solve the feasible problem P2.2 to obtain the continuous solution R^(w) and the Langrange multiplier Λ̄^(w) and calculate the infeasible cut;
        end if
        Solve the master problem P2.3 to obtain the binary vector Ψ^(w+1) and the lower bound LB;
        w=w+1;
    end while
    Update queue Q^i(t), Z^i(t);
    t=t+1;
end for
```

Table. I. prediction performance

| | DLinear | Prophet | GRU |
|---|---|---|---|
| MAE | **0.124** | 0.624 | 0.204 |
| MSE | **0.031** | 0.637 | 0.051 |
| RMSE | **0.175** | 0.798 | 0.225 |

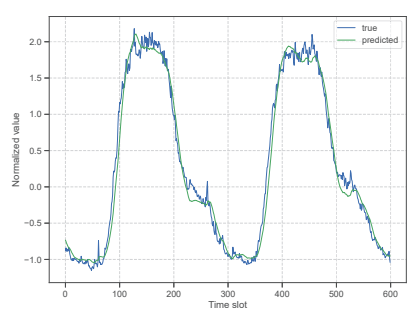


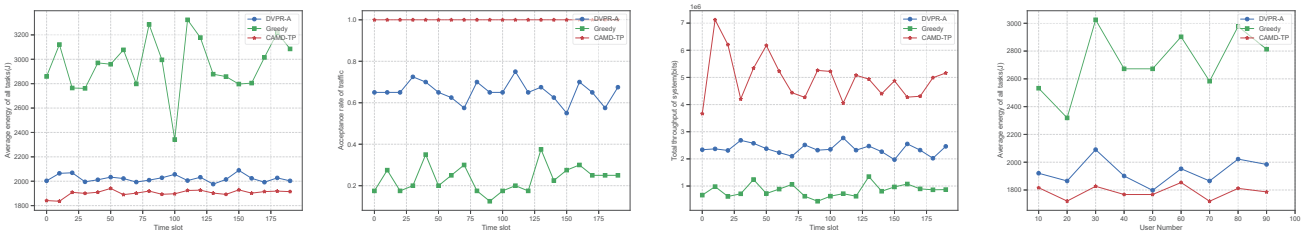

Fig. 2. Prediction result

### B. Algorithm Performance

We compare CAMD-TP with two heuristic algorithms. One is the Greedy algorithm, where each agent greedily selects the node with lowest energy consumption for migration. The



Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:16:05 UTC from IEEE Xplore. Restrictions apply.




(a) Energy consumption (b) Acceptance rate (c) Total throughput (d) Energy consumption vs. user number

Fig. 3. Algorithm Performance

other is DVPR-A [13], which constructs a generic heuristic optimization structure called VNF-Splited Multi-Stage edge-Weight Graph and solves it using a shortest path algorithm based on Lagrangian relaxation method. To ensure comparability, we only consider delay constraints and add a migration energy consumption component.

As illustrated in Figure. 3(a), CAMD-TP has the lowest average system energy consumption among the other two algorithms, achieving up to 18.7% total energy consumption reduction. CAMD-TP jointly optimizes the migration and resource allocation of all agents. However, the other two algorithms seek optimal results from a single-service perspective, resulting in higher energy consumption. Figure. 3(b) depicts a comparison of traffic acceptance rates over time. CAMD-TP ensures that the needs of all services are addressed. In contrast, DVPR-A and Greedy fail to consider the needs of all services simultaneously, resulting in later services being unable to find suitable migration solutions. Figure. 3(c) illustrates the traffic throughput comparison of the network. Since CAMD-TP has the highest service acceptance rate, it also has the highest throughput. Figure. 3(d) illustrates that as the number of users increases, CAMD-TP consistently maintains low average energy consumption, effectively optimizing resource management in the system and avoiding significant fluctuations.

## V. CONCLUSION

In this paper, we aim to minimize the long-term average system energy consumption and address the migration of AI service chains under service traffic fluctuation scenarios. It takes into full account the allocation of communication, computing, and memory resources in the network, as well as the QoS requirements of AI services, while minimizing system energy consumption. We utilize DLinear for accurate and rapid service traffic prediction and apply the Lyapunov optimization technique to convert the original long-term optimization problem into a mixed-integer programming problem. We propose an optimization algorithm based on GBD technique to effectively address the challenges of agent migration, DNN deployment and dynamic bandwidth resource allocation, ensuring efficient system operation in the face of dynamic changes. Experimental results show that, compared with two heuristic algorithms, CAMD-TP reduces the average system energy consumption up to 18.7%, and it outperforms existing algorithms in both service acceptance rate and throughput.

## ACKNOWLEDGMENTS

This work is funded by Beijing University of Posts and Telecommunications-China Mobile Research Institute Joint Innovation Center and the National Natural Science Foundation of China under Grant No.61971057.

## REFERENCES

[1] Y. Hu, Q. Li, Y. Chai, D. Wu, L. Lu, N. Shi, Y. Teng, and Y. Zhang, "Ai service deployment and resource allocation optimization based on human-like networking architecture," *IEEE Internet of Things Journal*, 2024.

[2] Y. Yue, X. Tang, W. Yang, Z. Zhang, and X. Zhang, "Virtual network function migration considering load balance and sfc delay in cloud datacenter," pp. 1–6, 2023.

[3] J. Chen, J. Chen, K. Guo, R. Hu, T. Zou, J. Zhu, H. Zhang, and J. Liu, "Fault tolerance oriented sfc optimization in sdn/nfv-enabled cloud environment based on deep reinforcement learning," *IEEE Transactions on Cloud Computing*, 2024.

[4] F. Zhang, H. Lu, F. Guo, and Z. Gu, "Traffic prediction based vnf migration with temporal convolutional network," in *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2021, pp. 1–6.

[5] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 9, 2023, pp. 11 121–11 128.

[6] A. Alleg, T. Ahmed, M. Mosbah, R. Riggio, and R. Boutaba, "Delay-aware vnf placement and chaining based on a flexible resource allocation approach," in *2017 13th international conference on network and service management (CNSM)*. ieee, 2017, pp. 1–7.

[7] H. Liu, C.-Z. Xu, H. Jin, J. Gong, and X. Liao, "Performance and energy modeling for live migration of virtual machines," in *Proceedings of the 20th international symposium on High performance distributed computing*, 2011, pp. 171–182.

[8] Y. Liu, J. Ran, H. Hu, and B. Tang, "Energy-efficient virtual network function reconfiguration strategy based on short-term resources requirement prediction," *Electronics*, vol. 10, no. 18, p. 2287, 2021.

[9] A. M. Geoffrion, "Generalized benders decomposition," *Journal of optimization theory and applications*, vol. 10, pp. 237–260, 1972.

[10] P. Cortez, M. Rio, M. Rocha, and P. Sousa, "Multi-scale internet traffic forecasting using neural networks and time series methods," *Expert Systems*, vol. 29, no. 2, pp. 143–155, 2012.

[11] S. Orlowski, R. Wessäly, M. Pióro, and A. Tomaszewski, "Sndlib 1.0—survivable network design library," *Networks: An International Journal*, vol. 55, no. 3, pp. 276–286, 2010.

[12] W. Rankothge, N. Gamage, H. Dewwiman, M. Ariyawansa, S. Suhail, and M. Senevirathne, "Network traffic prediction for a software defined network based virtualized network functions platform," in *2021 6th IEEE International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*, vol. 6. IEEE, 2021, pp. 1–4.

[13] L. Liu, S. Guo, G. Liu, and Y. Yang, "Joint dynamical vnf placement and sfc routing in nfv-enabled sdns," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4263–4276, 2021.



Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:16:05 UTC from IEEE Xplore. Restrictions apply.